

This assignment is concerned with the kinematic model of the 6-DOF myCobot 280 robot arm and the visualization of the robot arm in ROS 2 via RViz2. It therefore requires that you boot into Ubuntu in order to run ROS 2 Jazzy. Follow the steps of the following instruction files in the Moodle course, to install a dual boot with ROS 2 and prepare a workspace with the myCobot ROS 2 package:

- 1) Instructions on Ubuntu installation
- 2) Instructions on ROS 2 installation (Jazzy)
- 3) Instructions on myCobot package
- 4) Instructions on Jupyter

The final part of the assignment covers Jupyter–ROS 2 co-simulation of the robot.

Learning Objectives Forward Kinematics Lab

After completing the practical lab on *Forward Kinematics*, students will understand the fundamental principles, mathematical foundations, and computational techniques used to model and analyze the kinematics of robotic manipulators. They will gain the ability to derive and apply forward kinematic equations to determine the position and orientation of a robot's end-effector based on a given set of joint variables.

A central learning objective is to understand the mapping between the joint space and the task space of a manipulator. Students will learn how the joint configuration vector $\mathbf{q} = [q_1, q_2, \dots, q_n]^T$ is transformed into the end-effector pose ${}^b\mathbf{T}_e(\mathbf{q})$ represented as a homogeneous transformation matrix. They will be able to interpret this mapping both mathematically and geometrically and explain how the robot's structure and joint parameters influence its workspace and motion capabilities.

Students will become proficient in applying the Denavit-Hartenberg (DH) convention to systematically assign coordinate frames to robotic links and joints. They will learn how to identify and compute the four DH parameters ($d_i, \theta_i, a_i, \alpha_i$) for different manipulator types and use them to construct the sequence of homogeneous transformations that describe the robot's kinematic chain. They will understand the differences between standard and modified DH conventions and know how to implement both in practical applications.

The lab will also provide hands-on experience with modeling robotic manipulators in Python using Jupyter notebooks and common robotics libraries. Students will construct kinematic models of multi-link robots such as the myCobot 280 manipulator, define their joint types (revolute, prismatic, or fixed), and assemble them into a complete kinematic structure. They will learn to compute frame-to-frame transformations and visualize the resulting manipulator configuration in Python and ROS 2-based visualization tools like RViz2. Typical tooling includes NumPy/Matplotlib together with SpatialMath and the Robotics Toolbox for Python.

Furthermore, students will gain practical skills in simulating robot motion and interpreting kinematic data in co-simulation environments involving Jupyter and ROS 2. They will understand how to publish and subscribe to joint state information, synchronize transformations across different software components, and visualize the robot's motion in real time. Through these exercises, students will also develop a deeper understanding of how forward kinematics integrates into the broader robotics software stack, including robot description (URDF), simulation (Gazebo), and control interfaces.

By the end of the lab, students will be able to:

- Explain the concept and purpose of forward kinematics in robotic manipulation.
- Derive and apply forward kinematic equations to compute end-effector poses from joint variables.
- Use the Denavit-Hartenberg convention to parameterize robotic links and build transformation matrices.
- Model and simulate multi-DOF robotic arms using Python tooling (Jupyter, SpatialMath, Robotics Toolbox for Python).
- Implement and visualize kinematic models in ROS 2, including URDF-based robot descriptions.
- Integrate Jupyter-ROS 2 co-simulation to analyze and visualize robot motion and transformations.

These skills provide a fundamental basis for advanced robotics topics such as inverse kinematics, trajectory generation, and motion planning, and they are essential for designing and controlling robotic manipulators in research and industrial applications.

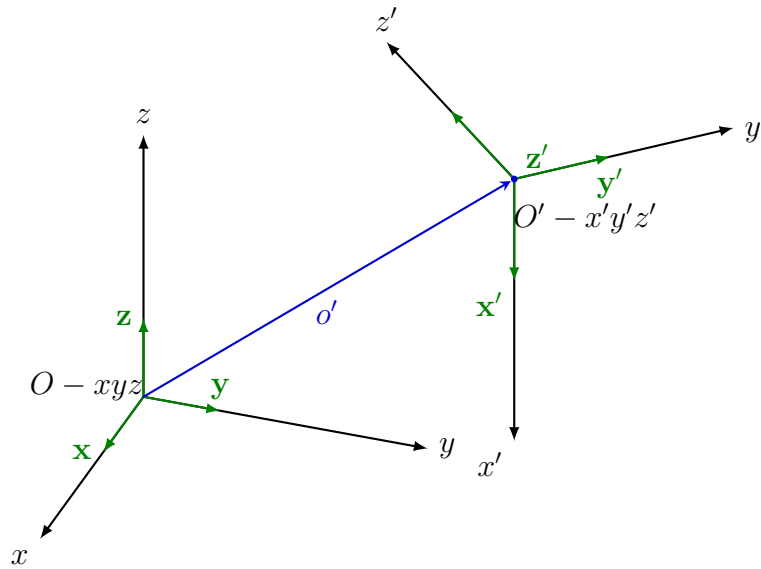


Figure 1: Rigid body transformation

Spatial Transformations

The pose of a rigid body in three-dimensional space is defined by its *position* and *orientation* with respect to a reference coordinate frame. In robotics, coordinate frames are attached to the links of a manipulator in order to describe their relative poses and motions.

Consider the two coordinate frames shown in Fig. 1. The frame $O - xyz$ serves as the reference frame, while the frame $O' - x'y'z'$ is translated and rotated with respect to $O - xyz$. The relative pose of $O' - x'y'z'$ with respect to $O - xyz$ is completely determined by a rotation matrix and a translation vector.

Rotation Matrices

The orientation of the frame $O' - x'y'z'$ with respect to the reference frame $O - xyz$ is represented by the rotation matrix

$$\mathbf{R} = [\mathbf{x}' \quad \mathbf{y}' \quad \mathbf{z}'], \quad (1)$$

where $\mathbf{x}'$, $\mathbf{y}'$, and $\mathbf{z}'$ denote the unit vectors along the axes of the primed coordinate frame expressed in the coordinates of the reference frame $O - xyz$. Consequently,

$$\mathbf{R} = \begin{bmatrix} x'_x & y'_x & z'_x \\ x'_y & y'_y & z'_y \\ x'_z & y'_z & z'_z \end{bmatrix}. \quad (2)$$

The columns of $\mathbf{R}$ therefore describe how the axes x' , y' , and z' of the primed frame are oriented with respect to the axes x , y , and z of the reference frame.

Since the coordinate axes form an orthonormal right-handed coordinate system, the rotation matrix belongs to the special orthogonal group

$$\mathbf{R} \in SO(3) \quad (3)$$

and satisfies

$$\mathbf{R}^T \mathbf{R} = \mathbf{R} \mathbf{R}^T = \mathbf{I}, \quad \det(\mathbf{R}) = 1. \quad (4)$$

It follows that the inverse of a rotation matrix is equal to its transpose,

$$\mathbf{R}^{-1} = \mathbf{R}^T. \quad (5)$$

Consider a vector $\mathbf{p}$ whose coordinates in the primed frame are denoted by $\mathbf{p}'$. If the origins of both coordinate frames coincide, the coordinates of the vector with respect to $O - xyz$ are obtained from

$$\mathbf{p} = \mathbf{R} \mathbf{p}'. \quad (6)$$

Thus, the rotation matrix transforms coordinates expressed in $O' - x'y'z'$ into coordinates expressed in $O - xyz$.

Translation Vectors

In addition to their relative orientation, the two coordinate frames in Fig. 1 have different origins. The position of the origin O' with respect to the origin O is described by the translation vector

$$\mathbf{o}' = \begin{bmatrix} o'_x \\ o'_y \\ o'_z \end{bmatrix}, \quad (7)$$

where the components of $\mathbf{o}'$ are expressed with respect to the reference frame $O - xyz$.

For a point P , let $\mathbf{p}'$ denote its coordinates with respect to $O' - x'y'z'$ and $\mathbf{p}$ its coordinates with respect to $O - xyz$. Considering both the rotation and the translation between the two frames gives

$$\boxed{\mathbf{p} = \mathbf{o}' + \mathbf{R} \mathbf{p}'} \quad (8)$$

The term $\mathbf{R} \mathbf{p}'$ rotates the coordinates from the primed frame into the orientation of the reference frame, while $\mathbf{o}'$ accounts for the displacement of the origin O' relative to O .

Homogeneous Transformation Matrices

Rotation and translation can be combined into a single matrix operation by introducing *homogeneous coordinates*. The pose of the frame $O' - x'y'z'$ with respect to the reference frame $O - xyz$ is represented by the homogeneous transformation matrix

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{o}' \\ \mathbf{0}^T & 1 \end{bmatrix}. \quad (9)$$

Using the individual components, this transformation can be written as

$$\mathbf{T} = \begin{bmatrix} x'_x & y'_x & z'_x & o'_x \\ x'_y & y'_y & z'_y & o'_y \\ x'_z & y'_z & z'_z & o'_z \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (10)$$

A three-dimensional point is represented in homogeneous coordinates by

$$\tilde{\mathbf{p}} = \begin{bmatrix} \mathbf{p} \\ 1 \end{bmatrix}, \quad \tilde{\mathbf{p}}' = \begin{bmatrix} \mathbf{p}' \\ 1 \end{bmatrix}. \quad (11)$$

The transformation between the two coordinate representations can then be written compactly as

$$\tilde{\mathbf{p}} = \mathbf{T} \tilde{\mathbf{p}}'. \quad (12)$$

The homogeneous transformation $\mathbf{T}$ therefore simultaneously describes the rotation and translation of the frame $O' - x'y'z'$ with respect to $O - xyz$.

The inverse transformation converts coordinates expressed in $O - xyz$ into coordinates expressed in $O' - x'y'z'$. It is given by

$$\mathbf{T}^{-1} = \begin{bmatrix} \mathbf{R}^T & -\mathbf{R}^T \mathbf{o}' \\ \mathbf{0}^T & 1 \end{bmatrix}. \quad (13)$$

Composition of Transformations

An important property of homogeneous transformations is that several transformations can be concatenated by matrix multiplication. If a third coordinate frame is defined relative to $O' - x'y'z'$, its pose with respect to $O - xyz$ can be obtained by multiplying the corresponding homogeneous transformation matrices.

This property is fundamental for serial robotic manipulators. A coordinate frame is assigned to each link of the kinematic chain, and the transformation from the robot base to the end-effector is obtained by successively multiplying the transformations between consecutive frames. Determining this transformation as a function of the joint variables constitutes the problem of *forward kinematics*.

Kinematic Chains and Robotic Manipulators

A robotic manipulator is a mechanical system composed of a sequence of rigid bodies, called *links*, which are connected by *joints*. The links form the mechanical structure of the robot, while the joints permit relative motion between adjacent links. The arrangement of links and joints defines the *kinematic structure* of the manipulator.

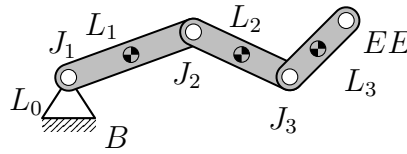


Figure 2: Kinematic chain

A joint constrains the relative motion between two connected links and introduces one or more degrees of freedom (DOF). The most common joint types in robotic manipulators are *revolute* and *prismatic* joints. A revolute joint permits rotation about a fixed joint axis, whereas a prismatic joint permits translation along a fixed joint axis. For a revolute joint, the corresponding joint variable q_i represents an angle, while for a prismatic joint it represents a linear displacement.

A sequence of links connected by joints is referred to as a *kinematic chain*. In a *serial manipulator*, the links and joints form a single chain from the robot base to the end-effector. Consequently, there exists a unique kinematic path from the base to each link and ultimately to the end-effector. A serial manipulator with n joints can be represented schematically as

$$\text{base} \rightarrow J_1 \rightarrow L_1 \rightarrow J_2 \rightarrow L_2 \rightarrow \cdots \rightarrow J_n \rightarrow L_n \rightarrow \text{end-effector}, \quad (14)$$

where J_i denotes joint i and L_i denotes link i .

The *base* represents the fixed part of the manipulator and provides the reference with respect to which the motion of the robot is described. The last link of the chain carries the *end-effector*. Depending on the application, the end-effector may be a gripper, a welding tool, a camera, or another device used to interact with the environment.

The configuration of the manipulator is determined by the positions of its joints. For a manipulator with n joints, the individual joint variables are collected in the *joint configuration vector*

$$\mathbf{q} = [q_1 \quad q_2 \quad \cdots \quad q_n]^T. \quad (15)$$

The set of all possible joint configurations is referred to as the *joint space* or *configuration space*. A manipulator with n independent one-degree-of-freedom joints therefore has n degrees of freedom. For example, the myCobot 280 considered in this lab consists of six revolute joints and consequently has six degrees of freedom.

An important property of a serial kinematic chain is that the motion of a joint affects all links located after this joint in the chain. A rotation of the first joint, for example, changes the pose of all subsequent links and therefore also the pose of the end-effector. In contrast, changing the last joint only affects the final part of the kinematic chain.

To mathematically describe the spatial arrangement of the links, coordinate frames are attached to the individual elements of the manipulator. A fixed *base frame* is associated with the robot base, while additional frames describe the position and orientation of the links along the kinematic chain. An *end-effector frame* is attached to the final link or tool.

The relative position and orientation between two consecutive coordinate frames can be represented by a homogeneous transformation matrix

$${}^{i-1}\mathbf{T}_i = \begin{bmatrix} {}^{i-1}\mathbf{R}_i & {}^{i-1}\mathbf{p}_i \\ \mathbf{0}^T & 1 \end{bmatrix}, \quad (16)$$

where ${}^{i-1}\mathbf{R}_i$ describes the orientation of frame i with respect to frame $i - 1$ and ${}^{i-1}\mathbf{p}_i$ describes the corresponding relative position.

For a serial manipulator, these transformations follow the structure of the kinematic chain and can be concatenated. The transformation from the base frame to the end-effector frame is therefore obtained as

$${}^0\mathbf{T}_n = {}^0\mathbf{T}_1 {}^1\mathbf{T}_2 \cdots {}^{n-1}\mathbf{T}_n. \quad (17)$$

Since the individual transformations depend on the joint variables, the position and orientation of the end-effector depend on the complete joint configuration $\mathbf{q}$. Determining this relationship between the joint configuration and the resulting end-effector pose is the objective of *forward kinematics*.

Forward Kinematics

A fundamental problem in robotics is to determine where a robot's end-effector is located for a given configuration of its joints. For a robotic manipulator, the position and orientation of the end-effector depend on the geometry of the robot and on the current joint variables. Knowledge of this relationship is essential for understanding and predicting the motion of a robot arm.

For example, when the joint angles of a manipulator are changed, the resulting position and orientation of the end-effector must be determined in order to visualize the robot configuration, analyze its workspace, plan motions, or control the robot. The mathematical relationship that maps the joint variables of a robot to the pose of its end-effector is referred to as *forward kinematics* or *direct kinematics*.

Forward kinematics therefore forms one of the fundamental building blocks of robotics and provides the basis for more advanced topics such as inverse kinematics, differential kinematics, trajectory generation, motion planning, and robot control.

Figure 3 illustrates the relationship between the *joint space* and the *work space* of a robotic manipulator. The configuration of a manipulator with n joints is described by the joint vector

$$\mathbf{q} = (q_1, \dots, q_n)^T, \quad (18)$$

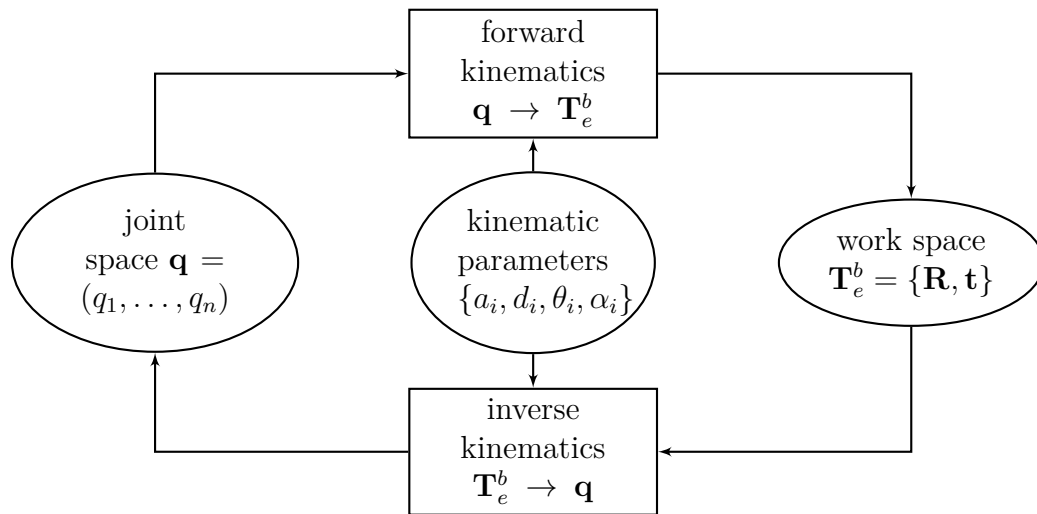


Figure 3: Forward and inverse kinematics

which defines a point in the joint space. Together with the kinematic parameters $\{a_i, d_i, \theta_i, \alpha_i\}$ describing the geometry of the robot, the joint configuration determines the pose of the end-effector.

The *forward kinematics* describes the mapping from joint space to work space,

$$\mathbf{q} \mapsto {}^b\mathbf{T}_e(\mathbf{q}), \quad (19)$$

where ${}^b\mathbf{T}_e$ denotes the homogeneous transformation from the end-effector frame e to the robot base frame b . It contains the orientation $\mathbf{R}$ and position $\mathbf{t}$ of the end-effector,

$${}^b\mathbf{T}_e = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix}. \quad (20)$$

Hence, forward kinematics answers the question: *Where is the end-effector for a given set of joint variables?* For a known robot geometry and a given joint configuration, the resulting end-effector pose is uniquely determined.

The *inverse kinematics* considers the opposite problem. Starting from a desired end-effector pose ${}^b\mathbf{T}_e$, it determines a joint configuration $\mathbf{q}$ that realizes this pose,

$${}^b\mathbf{T}_e \mapsto \mathbf{q}. \quad (21)$$

In contrast to forward kinematics, the inverse mapping is generally not unique. A desired end-effector pose may be reachable by several different joint configurations, may have a unique solution, or may not be reachable at all.

The kinematic parameters shown in the center of Fig. 3 establish the geometric relationship between both spaces. They describe the dimensions and relative arrangement of

the links and joints and are therefore required for both forward and inverse kinematics. In this lab, these parameters are represented using the Denavit–Hartenberg convention introduced in the following section.

Forward Kinematics of Serial Robotic Manipulator

Consider a serial robotic manipulator with n joints. Its configuration is described by the joint vector

$$\mathbf{q} = [q_1 \quad q_2 \quad \cdots \quad q_n]^T, \quad (22)$$

where q_i represents the variable of joint i . For a revolute joint, q_i corresponds to a joint angle, whereas for a prismatic joint it represents a linear displacement.

The objective of forward kinematics is to determine the pose of the end-effector with respect to the robot base frame from the joint configuration $\mathbf{q}$. This defines the mapping

$$\mathbf{q} \mapsto {}^b\mathbf{T}_e(\mathbf{q}), \quad (23)$$

where ${}^b\mathbf{T}_e$ is the homogeneous transformation from the end-effector frame e to the base frame b . It can be written as

$${}^b\mathbf{T}_e = \begin{bmatrix} {}^b\mathbf{R}_e & {}^b\mathbf{p}_e \\ \mathbf{0}^T & 1 \end{bmatrix}, \quad (24)$$

where ${}^b\mathbf{R}_e \in SO(3)$ describes the orientation of the end-effector and ${}^b\mathbf{p}_e \in \mathbb{R}^3$ describes its position relative to the robot base.

For a serial manipulator, the transformation from the base to the end-effector is obtained by successively combining the transformations between consecutive links,

$${}^b\mathbf{T}_e(\mathbf{q}) = {}^0\mathbf{T}_1(q_1) {}^1\mathbf{T}_2(q_2) \cdots {}^{n-1}\mathbf{T}_n(q_n). \quad (25)$$

Consequently, forward kinematics describes how the individual joint motions propagate through the kinematic chain and determine the resulting end-effector pose in task space.

Denavit–Hartenberg Convention

The forward kinematics of a serial manipulator can be obtained by successively multiplying the homogeneous transformations between consecutive joint frames. The *Denavit–Hartenberg (DH) convention* provides a systematic procedure for assigning these coordinate frames and describing each transformation with only four parameters. In this way, the geometry of a serial kinematic chain can be represented in a compact and standardized form.

Figure 4 illustrates the assignment of coordinate frames and the DH parameters for two consecutive links. According to the standard DH convention, the z_{i-1} -axis is aligned with

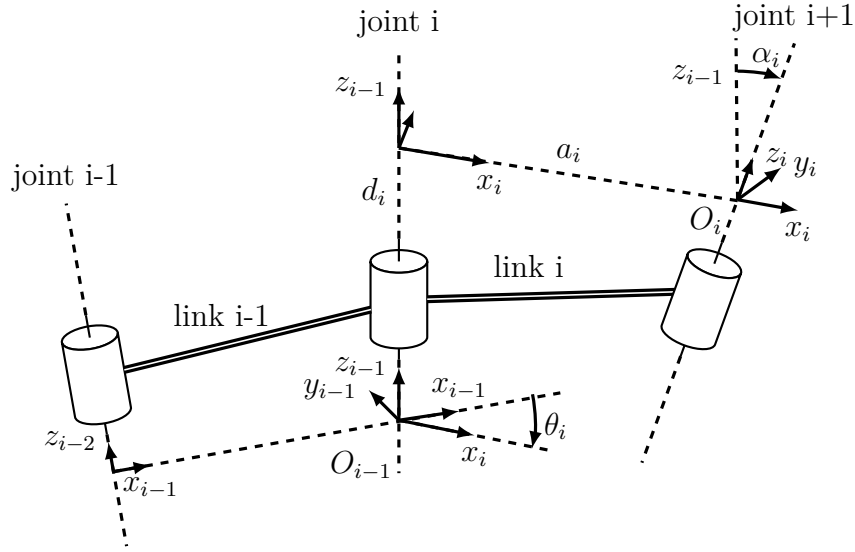


Figure 4: Denavit-Hartenberg parameter

the axis of joint i , while the z_i -axis is aligned with the axis of joint $i + 1$. The x_i -axis is chosen along the common normal between the two consecutive joint axes z_{i-1} and z_i . The origin O_i is located at the intersection of this common normal with the z_i -axis. Finally, the y_i -axis is chosen such that x_i , y_i , and z_i form a right-handed coordinate system.

With this assignment, the relative pose of frame i with respect to frame $i - 1$ is completely described by the four DH parameters

$$d_i, \quad \theta_i, \quad a_i, \quad \alpha_i. \quad (26)$$

Their geometric interpretation can be directly identified in Fig. 4. The parameter d_i is the distance measured along z_{i-1} from the origin O_{i-1} to the common normal x_i . The angle θ_i describes the rotation about z_{i-1} from x_{i-1} to x_i . The parameter a_i is the distance between the two joint axes z_{i-1} and z_i , measured along x_i . Finally, α_i is the angle between z_{i-1} and z_i , measured as a rotation about x_i .

The transformation from frame $i - 1$ to frame i can therefore be constructed as a sequence of four elementary transformations,

$${}^{i-1}\mathbf{T}_i = \text{Trans}_z(d_i) \text{Rot}_z(\theta_i) \text{Trans}_x(a_i) \text{Rot}_x(\alpha_i). \quad (27)$$

Using homogeneous transformation matrices, this results in

$${}^{i-1}\mathbf{T}_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (28)$$

For a revolute joint, the joint motion is represented by the angle θ_i , while d_i remains constant. Thus, the joint variable q_i determines θ_i , possibly including a constant joint offset,

$$\theta_i = q_i + \theta_{i,0}. \quad (29)$$

For a prismatic joint, the situation is reversed: θ_i is constant and the displacement d_i represents the joint variable,

$$d_i = q_i + d_{i,0}. \quad (30)$$

Once the DH parameters have been determined for all joints, the complete forward kinematics follows from concatenating the individual transformations along the kinematic chain. For a serial manipulator with n joints, the end-effector pose with respect to the base frame is

$${}^0\mathbf{T}_n(\mathbf{q}) = {}^0\mathbf{T}_1(q_1) {}^1\mathbf{T}_2(q_2) \cdots {}^{n-1}\mathbf{T}_n(q_n). \quad (31)$$

The DH convention therefore reduces the description of the complete kinematic structure to a table containing four parameters for each joint. Determining these parameters from the geometry of the manipulator is the central step in establishing its forward kinematic model.

Robot Operating System 2

The *Robot Operating System 2* (*ROS 2*) is a software framework for developing robotic applications. It provides a standardized infrastructure for organizing a robotic system into individual software components and for exchanging information between these components. Instead of implementing a robotic application as one monolithic program, ROS 2 encourages a distributed and modular architecture in which individual components perform dedicated tasks and communicate through well-defined interfaces.

A robotic application may, for example, contain separate components for reading sensors, estimating the robot state, planning a trajectory, controlling the robot joints, and visualizing the resulting motion. ROS 2 provides the communication infrastructure that allows these components to exchange data, even if they are implemented in different programming languages or run in different processes.

The fundamental computational unit of a ROS 2 application is a *node*. A node is a running software component that performs a specific task. A robot system typically consists of several nodes that operate simultaneously. For example, one node may provide the current joint configuration of a manipulator, while another node computes desired joint positions and a third node visualizes the robot.

The collection of nodes and their communication connections forms the *ROS graph*. ROS 2 provides command-line tools for inspecting this graph. For example,

```
ros2 node list
```

displays the currently active nodes, while

```
ros2 topic list
```

displays the available communication topics.

In addition to nodes and topics, ROS 2 provides different communication mechanisms such as services and actions. For the exchange of continuously generated data, however, the most important mechanism is communication through *topics* according to the publisher–subscriber principle.

Publisher–Subscriber Architecture

The publisher–subscriber architecture allows ROS 2 nodes to exchange information without requiring a direct connection between the communicating nodes. Instead, communication is organized using named channels called *topics*.

A node that generates information can act as a *publisher*. The publisher sends data to a particular topic. A node that requires this information can create a *subscriber* to the same topic and receives the published data. A topic can have multiple publishers and multiple subscribers.

The basic communication principle can therefore be represented as

$$\text{Publisher Node} \xrightarrow{\text{Topic}} \text{Subscriber Node.} \quad (32)$$

The publisher does not need to know which nodes receive its data, and the subscriber does not need to know which particular node generated it. Both components only have to agree on the name of the topic and the type of data exchanged. This decoupling is an important aspect of the modular architecture of ROS 2.

For example, the configuration of a robotic manipulator can be distributed using the topic

```
/joint_states
```

A node providing the current joint configuration publishes messages to this topic. Other nodes interested in the robot configuration can subscribe to `/joint_states`. A visualization component such as RViz2 can then use the received joint states together with the robot description to visualize the current configuration of the manipulator.

Communication via topics is particularly suitable for information that is generated continuously or repeatedly, such as joint positions, sensor measurements, camera images, velocities, or estimated robot poses.

Topics and Messages

A *topic* is a named communication channel within the ROS 2 graph. The topic name describes the information exchanged through the channel. Examples of typical topic names are

```
/joint_states  
/camera/image  
/cmd_vel
```

Topics do not define the actual data themselves. The structure of the transmitted data is specified by a *message type*. Every publisher and subscriber communicating through a particular topic must use compatible message types.

A ROS 2 *message* is a structured data object consisting of typed fields. Depending on the message type, these fields may contain scalar values, vectors, strings, time stamps, or other messages. ROS 2 provides standardized message types for many common robotics applications.

For representing the state of robot joints, ROS 2 provides the message type

```
sensor_msgs/msg/JointState
```

which contains, among others, the fields

```
name  
position  
velocity  
effort
```

The **name** field identifies the joints, while **position** contains the corresponding joint variables. For a revolute joint, the position represents a joint angle, whereas for a prismatic joint it represents a linear displacement. The message additionally contains a time stamp that identifies when the joint state was recorded.

The association between a topic and its message type can be inspected using

```
ros2 topic info /joint_states
```

and the structure of a particular message type can be displayed using

```
ros2 interface show sensor_msgs/msg/JointState
```

Messages currently transmitted through a topic can be inspected directly from the command line. For example,

```
ros2 topic echo /joint_states
```

displays the joint-state messages as they are received.

Publishers and Subscribers in Python

ROS 2 nodes can be implemented in Python using the `rclpy` client library. A Python node is typically derived from the ROS 2 Node class,

```
import rclpy
from rclpy.node import Node

class RobotNode(Node):

    def __init__(self):
        super().__init__('robot_node')
```

A publisher is created by specifying the message type, topic name, and a Quality-of-Service queue depth. For example, a publisher for joint states can be created as

```
self.publisher = self.create_publisher(
    JointState,
    '/joint_states',
    10
)
```

A message is subsequently transmitted by calling

```
self.publisher.publish(msg)
```

on the publisher.

A subscriber is created in a similar manner,

```
self.subscription = self.create_subscription(
    JointState,
    '/joint_states',
    self.joint_state_callback,
    10
)
```

where `joint_state_callback` denotes a callback function. Whenever a new message is received on the subscribed topic, ROS 2 invokes this function and passes the received message to it,

```
def joint_state_callback(self, msg):
    q = msg.position
```

The publisher and subscriber therefore operate asynchronously. The publisher generates messages whenever new information is available, while the subscriber reacts whenever a corresponding message arrives.

ROS 2 Communication in a Robotic Manipulator

For a robotic manipulator, the publisher–subscriber architecture provides a convenient interface between the mathematical robot model, simulation, controllers, sensors, and visualization tools. The joint configuration

$$\mathbf{q} = [q_1 \quad q_2 \quad \cdots \quad q_n]^T \quad (33)$$

used in the kinematic model can be communicated between ROS 2 nodes using appropriate messages.

A typical information flow for the state of a manipulator is

$$\text{Robot / Simulation} \longrightarrow \text{/joint_states} \longrightarrow \text{Robot State / Visualization.} \quad (34)$$

In the opposite direction, desired joint configurations or trajectories can be published to a controller,

$$\text{Planning / Python} \longrightarrow \text{Joint Command Topic} \longrightarrow \text{Robot Controller.} \quad (35)$$

ROS 2 thus provides the communication layer that connects the mathematical description of a robot with its simulation and control software. In this lab, these concepts are applied to the myCobot 280 in order to exchange joint information and visualize or command different configurations of the manipulator.

Task Description

The objective of this lab is to derive, implement, and visualize the forward kinematics of the six-degree-of-freedom *myCobot 280* robotic manipulator. The Denavit–Hartenberg (DH) convention is used to systematically describe the geometry of the robot and the transformations between consecutive joint frames.

First, coordinate frames are assigned to the links of the manipulator and the corresponding DH parameters are identified. From these parameters, homogeneous transformation matrices are constructed and combined to obtain the transformation from the robot base to the end-effector.

The resulting kinematic model is subsequently implemented in Python using a Jupyter notebook and robotics libraries such as SpatialMath and the Robotics Toolbox for Python. The implementation is used to compute and analyze transformations for different joint configurations.

In the second part of the lab, the kinematic description is related to the Unified Robot Description Format (URDF) model of the myCobot 280. The robot configuration and the corresponding coordinate transformations are visualized in ROS 2 using RViz2. Finally, Python and ROS 2 are combined in a co-simulation in which joint configurations generated in Python are communicated to ROS 2 via the `/joint_states` topic and the resulting robot motion is visualized in RViz2.

The lab therefore establishes the connection between the mathematical description of forward kinematics, its computational implementation, and its practical representation within a modern robotic software framework.

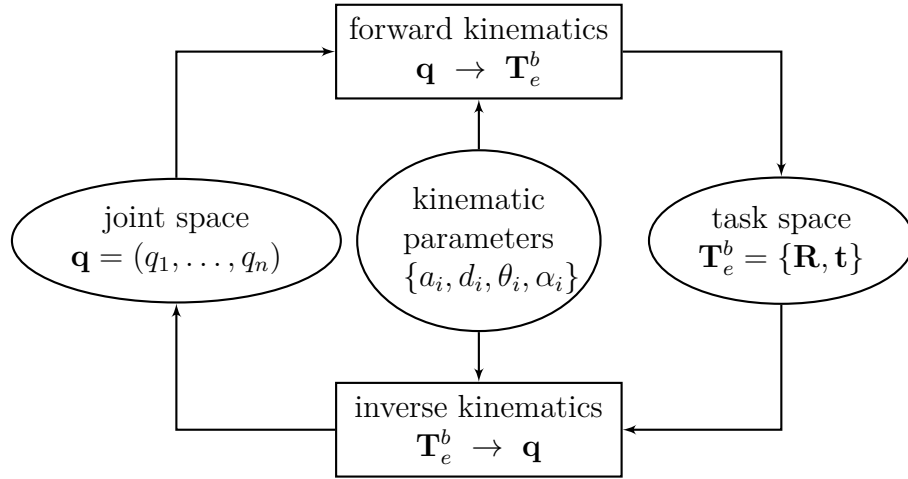


Figure 5: Joint space and task space coordinates

Direct Kinematics

Direct or forward kinematics utilizes the kinematic equations of a robot to compute the end-effector pose in task space from the joint configuration. More precisely it establishes a mapping between the joint vector $\mathbf{q} = [q_1, \dots, q_n]^T$ and the homogeneous transform $\mathbf{T}_e^b(\mathbf{q})$ that describes the end-effector position and orientation w.r.t. the robot base frame. Figure 5 illustrates the mapping between joint space and task via the forward kinematics and the reverse relationship mediated by inverse kinematics. The forward kinematics equations not only depend on the (dynamic) joint configuration $\mathbf{q} = [q_1, \dots, q_n]^T$ but also on the set of parameters $\{a_i, d_i, \theta_i, \alpha_i\}$ that define the geometry of the sequence of links and joints.

The kinematics equations of a serial link robot arm are obtained from a sequence of transformations that characterize the relative movement at each joint. In case of a revolute joint, the joint variable q_i denotes the angle ϑ_i between the links. In case of a prismatic joint, the joint variable q_i denotes the linear displacement d_i between links. The result is a sequence of rigid transformations T_i between consecutive robot joints from the base frame to the end effector frame parameterized by the joint angles $\mathbf{q} = [q_1, \dots, q_n]^T$.

Denavit-Hartenberg Convention

The Denavit-Hartenberg (DH) convention defines a procedure to uniquely define the joint frames and the corresponding homogeneous transformations in terms of four parameters per joint. The convention standardizes the frames for the geometric linkage among joints.

The Denavit-Hartenberg convention represents the homogeneous transformation between the consecutive joint frames i and $i + 1$ as a sequence of four elemental transformations, associated with four parameters $d_i, \theta_i, a_i, \alpha_i$.

- d_i denotes the offset from O_{i-1} to the common normal (line perpendicular to both z axes) along z_{i-1} . In case of a prismatic joint d_i constitutes the joint variable q_{i-1} .

- θ_i is the angle between x_{i-1} and x_i about the z_{i-1} -axis. In case of a revolute joint θ_i constitutes the joint variable q_{i-1} .
- a_i denotes the length of the common normal. Assuming a revolute joint, a is the radius about z_{i-1} .
- α_i is the angle between z_{i-1} and z_i about the x_i -axis (parallel to the common normal).

In order to apply the standard DH procedure, the following properties of the coordinate frames are required:

- The z -axis is the direction of the joint axis.
- The x -axis is parallel to the common normal $x_i = z_{i-1} \times z_i$ and directs from z_{i-1} to z_i . If both z axes are parallel, then d is a free parameter.
- The y -axis follows from the x - and z -axis by choosing it to be a right-handed coordinate system.

Notice, that the literature employs two different Denavit-Hartenberg conventions: the standard convention mentioned above and a modified DH convention in which the locations of coordinate systems and the order of performed transformations differ (modified: $a_i, \alpha_i, d_i, \theta_i$).

A few words on joint offsets: In any DH convention, the joint variables may have arbitrary values. If you wish to define the kinematics drawing with the associated coordinate frames to be a meaningful reference configuration (e.g. all joint variables are zero), you have to define joint offsets. The joint offsets usually do not count to the DH parameters and are thus omitted in DH tables found in literature.

Denavit-Hartenberg Parameters myCobot 280

Figure 6 shows the myCobot 280 robot arm with the assigned joint frames in its reference configuration. Table 1 summarizes the Denavit-Hartenberg (standard DH) parameters used in this lab. Notice that, in addition to the DH parameters $(a_i, d_i, \alpha_i, \theta_i)$, the model includes joint *offsets*. These offsets implement the reference configuration convention used by the robot model and are applied as $\theta_i = q_i + \text{offset}_i$ for revolute joints.

Proceed with tasks 1 and 2 in the template.

Modeling myCobot 280 Robot in Python/Jupyter

Note: The lab is solved in Jupyter notebooks using Python. The recommended libraries are NumPy/Matplotlib together with SpatialMath and the Robotics Toolbox for Python.

Robot Model in Python

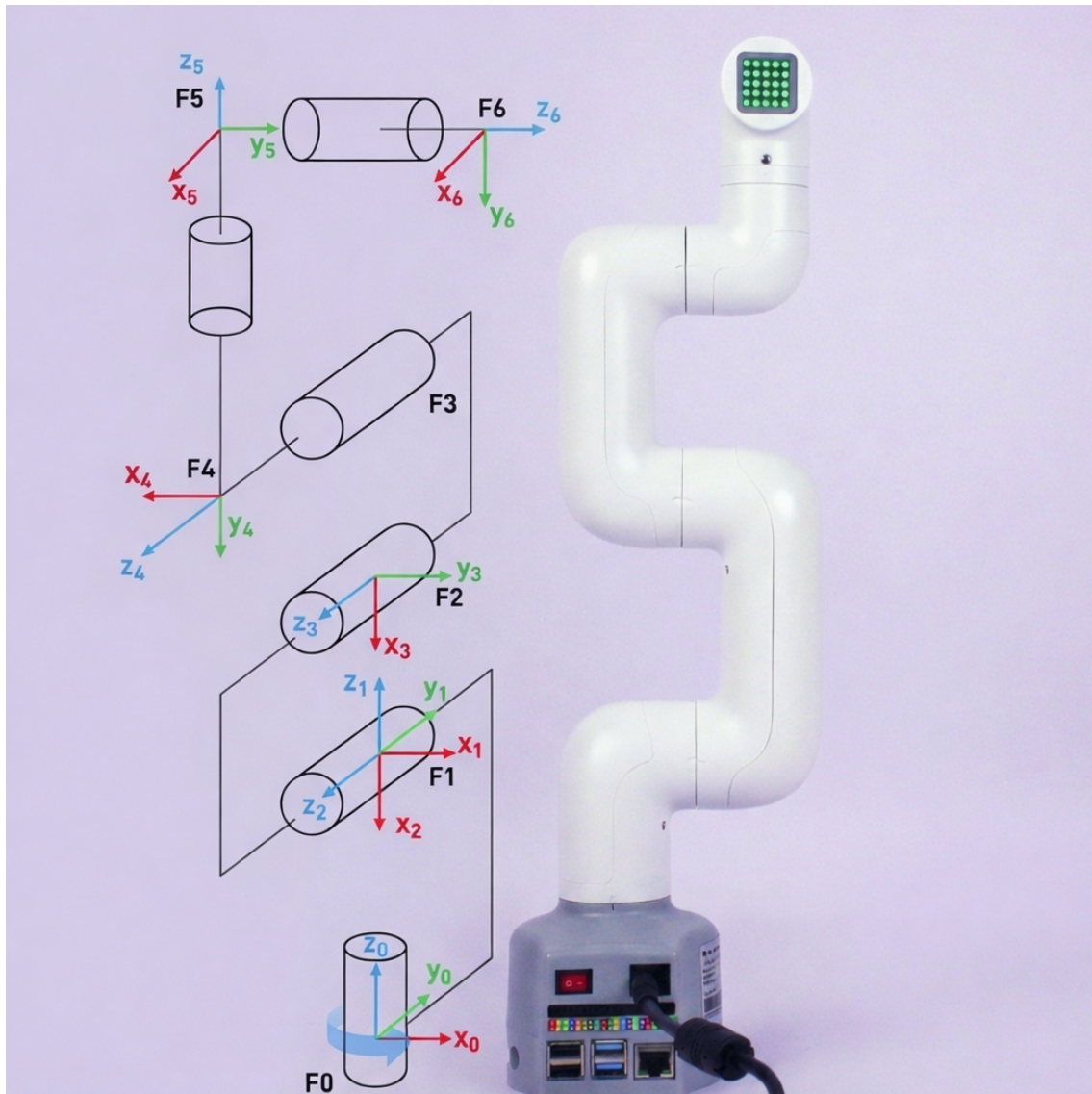


Figure 6: myCobot 280 robot with joint frames

In this lab you will build and use a serial-link robot model in Python. A convenient approach is to use a DH-based robot model with the Robotics Toolbox for Python, together with SpatialMath for homogeneous transformations.

Typical imports in the notebook are:

```
import numpy as np
from spatialmath import SE3
from spatialmath.base import tr2rpy
import roboticstoolbox as rtb
```

A DH-based robot can be created by defining its links (example syntax shown here; the template provides the myCobot 280 model and parameters):

i	θ_i	d_i [m]	a_i [m]	α_i [rad]
1	q_1	0.13122	0	1.5708
2	q_2	0	-0.1104	0
3	q_3	0	-0.096	0
4	q_4	0.0634	0	1.5708
5	q_5	0.07505	0	-1.5708
6	q_6	0.0456	0	0

Table 1: Denavit–Hartenberg parameters (standard DH) for myCobot 280.

i	offset _{i} [rad]
1	0
2	-1.5708
3	0
4	-1.5708
5	1.5708
6	0

Table 2: Joint offsets for the myCobot 280 DH model used in this lab.

```
# Example pattern (the template contains the concrete myCobot model)
L1 = rtb.RevoluteDH(a=..., d=..., alpha=..., offset=...)
# ...
robot = rtb.DHRobot([L1, L2, L3, L4, L5, L6], name="myCobot280")
```

Joint Configurations and Transforms

The joint configuration is described by a vector of joint variables $\mathbf{q}$ (in radians for revolute joints). With the robot model you are able to calculate various transforms between robot link frames for a given joint configuration.

Forward kinematics (end-effector pose) is obtained with:

```
T = robot.fkine(q)      # SE3 object
T.A                      # 4x4 homogeneous matrix
```

To obtain intermediate link transforms (frame-to-frame), the template notebook provides helper code that evaluates the individual DH transformations and/or extracts link frames from the model.

Proceed with tasks 3 to 8 in the template.

Notes:

- Be careful when running sections twice, as you may accidentally redefine the robot model or overwrite variables in the notebook.

- The printed robot structure in the notebook (link and joint listing) should match the model definition provided in the template.

Robot Description Using URDF

The *Unified Robot Description Format (URDF)* is an XML-based format used in ROS to describe the structure and physical properties of a robot. A URDF model provides a common representation of the robot that can be used by different software components for visualization, simulation, kinematic computations, motion planning, and control. Instead of defining the geometry and kinematic structure separately in each application, the robot description can therefore serve as a central model of the robotic system.

A URDF model represents a robot as a tree consisting primarily of *links* and *joints*. Links represent the rigid bodies of the robot, whereas joints define the kinematic relationships between these bodies. For a serial manipulator, this results in a chain of alternating links and joints, starting at the base and ending at the end-effector,

$$\text{base link} \rightarrow \text{joint}_1 \rightarrow \text{link}_1 \rightarrow \cdots \rightarrow \text{joint}_n \rightarrow \text{end-effector link}. \quad (1)$$

The resulting link–joint structure directly represents the kinematic chain of the manipulator.

Links

A **link** element describes a rigid body of the robot. Each link has its own coordinate frame and can contain information about its visual appearance, collision geometry, and physical properties. A simplified link definition may have the form

```
<link name="link_1">
<visual>
...
</visual>
<collision>
...
</collision>
<inertial>
...
</inertial>
</link>
```

The **visual** element specifies how the link is displayed, for example by defining a primitive geometry or referencing a three-dimensional mesh file. The **collision** element defines the geometry that can be used for collision detection. The **inertial** element contains physical properties such as the mass, center of mass, and inertia tensor, which are required for dynamic simulation.

The visual and collision models do not necessarily have to be identical. Detailed mesh models are often used for visualization, while simpler geometries can be employed for collision detection to reduce computational effort.

Joints

A **joint** element defines the kinematic connection between two links. Each joint specifies a *parent link* and a *child link*. It also defines the position and orientation of the joint frame relative to its parent and, for movable joints, the joint axis and possible motion limits.

A typical revolute joint can be represented as

```
<joint name="joint_1" type="revolute">
  <parent link="base_link"/>
  <child link="link_1"/>

  <origin xyz="0 0 0.13122"
    rpy="0 0 0"/>

  <axis xyz="0 0 1"/>

  <limit lower="-3.14"
    upper="3.14"
    effort="..."
    velocity="..."/>
</joint>
```

The **origin** element specifies the fixed transformation between the parent-link frame and the joint frame. Its **xyz** attribute defines the translation, while **rpy** specifies the orientation using roll, pitch, and yaw angles. The **axis** element specifies the direction of the joint axis in the joint coordinate frame.

The joint **type** determines the permitted relative motion between the connected links. Important joint types for robot manipulators include **revolute**, **continuous**, **prismatic**, and **fixed**. A revolute joint rotates about its specified axis within given limits, while a continuous joint permits unrestricted rotation. A prismatic joint produces a translation along its joint axis. A fixed joint introduces no degree of freedom and rigidly connects two links.

For a revolute joint, its joint variable q_i describes a rotation about the specified axis. For a prismatic joint, q_i describes a translation along the axis. Consequently, the joint values

$$\mathbf{q} = [q_1 \quad q_2 \quad \cdots \quad q_n]^T \quad (2)$$

together with the fixed transformations specified in the URDF determine the configuration of the complete kinematic chain.

Kinematic Information

The combination of the `parent`, `child`, `origin`, and `axis` elements defines the kinematic structure of the robot. Each joint introduces a transformation between two neighboring links. For a revolute joint, this transformation can conceptually be written as

$${}^{i-1}\mathbf{T}_i(q_i) = {}^{i-1}\mathbf{T}_{J_i}\mathbf{T}_{J_i}(q_i), \quad (3)$$

where ${}^{i-1}\mathbf{T}_{J_i}$ represents the constant translation and rotation specified by the joint's `origin` element and $\mathbf{T}_{J_i}(q_i)$ represents the variable transformation generated by the joint motion.

For a serial manipulator, the end-effector transformation can then be obtained by concatenating the transformations along the kinematic tree,

$${}^b\mathbf{T}_e(\mathbf{q}) = {}^b\mathbf{T}_1(q_1){}^1\mathbf{T}_2(q_2) \cdots {}^{n-1}\mathbf{T}_n(q_n). \quad (4)$$

Thus, a URDF description contains sufficient structural information to construct a kinematic model without manually specifying a Denavit–Hartenberg table. It should be noted that URDF and the DH convention are different representations of the same underlying kinematic structure. URDF describes transformations between links and joints directly, whereas the DH convention uses a particular systematic assignment of coordinate frames and four parameters per transformation.

URDF in ROS 2

Within ROS 2, the URDF model is commonly made available to other nodes through the `robot_description` parameter. The `robot_state_publisher` uses the robot description together with the current joint values to calculate the coordinate transformations between the individual robot links.

The current values of movable joints are typically provided through the

`/joint_states`

topic. Combining these joint values with the kinematic information contained in the URDF allows the pose of every link to be determined. The resulting coordinate transformations can be published to the ROS 2 TF system and used, for example, by RViz2 to display the current configuration of the robot.

The relationship can therefore be summarized as

$$\begin{aligned} \text{URDF} + \text{joint configuration } \mathbf{q} \\ \longrightarrow \text{link transformations} \longrightarrow \text{TF} \longrightarrow \text{robot visualization.} \end{aligned} \quad (5)$$

The URDF provides the fixed structure and geometry of the robot, while the joint-state messages provide the time-varying configuration.

URDF Model of the myCobot 280

In this lab, the URDF description of the six-degree-of-freedom *myCobot 280* is used to construct the kinematic robot model. The URDF specifies the sequence of links and revolute joints, their relative transformations, joint axes, and joint limits. It additionally provides the geometric information required for visualization of the manipulator.

The same robot description can be used by different components of the experimental environment. ROS 2 uses the model to represent the robot and its coordinate frames, while the Robotics Toolbox for Python can import the URDF and construct a corresponding kinematic model. The latter provides the forward kinematic mapping

$$\mathbf{q} \mapsto {}^b\mathbf{T}_e(\mathbf{q}), \quad (6)$$

which is subsequently required by the algorithms used in the Jupyter notebook.

Using the URDF therefore connects the different parts of the lab: the same structural robot description forms the basis for kinematic calculations in Python, the representation of the robot in ROS 2, and the visualization of its configuration in RViz2.

Unified Robot Description Format

Unified Robot Description Format (URDF) is an XML format for representing a robot model. The `urdf_to_graphviz` tool generates a graphviz diagram of the URDF model including all components. The blue ellipses denote joints and the black boxes denote robot links. The transform between links is stated in terms of a translation vector *xyz* and a rotation expressed in terms of roll pitch yaw angles *rpy*.

A URDF-file contains the geometric (kinematic) and dynamic parameters of the links (bodies) and joints. It also refers to the CAD data for visualization of bodies in RViz2 or Gazebo.

In this lab, you will use the URDF provided by the myCobot 280 ROS 2 package for visualization in RViz2.

The following code describes a link in the file `mycobot_280_gazebo.urdf`. It specifies geometry, visual mesh data, collision geometry and inertial parameters:

```
<link name="g_base">
  <visual>
    <geometry>
      <mesh filename="package://mycobot_description/urdf/mycobot_280_pi/G_base.dae"/>
    </geometry>
    <origin xyz="0 0 -0.03" rpy="0 0 1.5708"/>
  </visual>
  <collision>
```

```
<geometry>
  <box size="0.08 0.08 0.04"/>
</geometry>
<origin xyz="0 0 -0.03" rpy="0 0 1.5708"/>
</collision>
<inertial>
  <mass value="0.33"/>
  <inertia ixx="0.000784" ixy="0" ixz="0" iyy="0.000867" iyz="0" izz="0.001598"/>
  <origin xyz="0 0 0" rpy="0 0 0"/>
</inertial>
</link>
```

The following code defines a revolute joint and specifies the transform associated with the joint in terms of translation xyz and roll-pitch-yaw angles rpy , as well as axis and joint limits:

```
<joint name="joint4_to_joint3" type="revolute">
  <parent link="joint3"/>
  <child link="joint4"/>
  <origin xyz="-0.1104 0 0" rpy="0 0 0"/>
  <axis xyz="0 0 1"/>
  <limit lower="-2.618" upper="2.618" effort="5.0" velocity="2.0"/>
  <dynamics damping="0.05" friction="0.01"/>
</joint>
```

Proceed with tasks 9 to 16 in the template.

Python–ROS 2 Cosimulation of myCobot 280 Robot

RViz2

RViz2 is the standard ROS 2 tool for visualization of robot related data. RViz2 provides a number of different camera perspectives so called views to visualize the environment. A display in RViz2 is something that draws data in the 3D world such as a point cloud or the robot state. The RViz2 user guide explains the GUI and the functionalities that RViz2 provides. The built-in display `RobotModel` shows a visual representation of the robot according to current joint and link TF transforms.

myCobot ROS 2 Package

The myCobot repository contains packages and files for installation/usage of the Elephant Robotics myCobot. Refer to the Moodle workspace to find instructions on how to install this package and its dependencies. The package supports pure visualization (RViz2), simulation (Gazebo) and (depending on setup) control interfaces.

A typical launch file for visualization starts RViz2 with a predefined setup:

```
ros2 launch <mycobot_package> <view_launch_file>.launch.py
```

Please inspect the topics which are made available by the package.

Proceed with tasks 17 and 18 in the template.

Connect to ROS 2 and List Topics

You can skip this introduction to basic ROS 2 concepts such as topics, publishers, subscribers and messages in Python in case you are already familiar with ROS 2 and `rclpy`.

Launching and interacting a robotic application in ROS 2 typically involves these steps:

- Start a ROS 2 node in Python using `rclpy`.
- Exchange data through publishers and subscribers (topics) using typed ROS 2 messages.

The command `ros2 node list` lists all the nodes that are currently available on the ROS 2 network. Similarly `ros2 topic list` reports the topics that are currently available on the ROS 2 network. With `ros2 topic info <topic>` you obtain specific information about a particular topic.

Proceed with tasks 19 to 21 in the template.

ROS 2 Concepts

The runtime architecture of ROS 2 is composed of these components: Nodes, services and topics.

Nodes: Nodes are processes that compute information in a modular fashion. Each node constitutes a running instance of some small ROS 2 program. Robot controllers in ROS 2 are modular comprised of multiple nodes for designated tasks such as processing sensor raw data, actuation of robot motors, localization and path planning.

Messages: Nodes communicate with each other by passing messages. A message is a data structure (similar to C/C++ or Python objects) organized in typed fields. ROS 2 supports primitive data types (integer, float, boolean, etc.) as well as arrays of primitive types.

Topic: Messages in ROS 2 are organized into named topics. A topic is a unique name that indicates the content of a message. This means that a topic type is defined by the message type published on it.

ROS 2 Subscriber and Publisher

The basic communication mechanism for ROS 2 nodes are messages. Messages are structured into named topics. Any node that shares his information publishes messages on the appropriate topic and those nodes which are interested in that information subscribe to the topic.

Our simulation utilizes the topic `/joint_states` with the message type `sensor_msgs/msg/JointState` to communicate the myCobot 280 joint configuration `q` between different nodes. The joint names in the URDF used in this lab are:

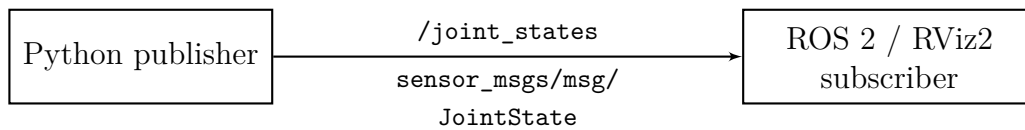


Figure 7: Python–ROS 2 co-simulation of myCobot 280 via the `/joint_states` topic

```

joint2_to_joint1, joint3_to_joint2, joint4_to_joint3, joint5_to_joint4,
joint6_to_joint5, joint6output_to_joint6
  
```

The message type `sensor_msgs/msg/JointState` encodes the joint configuration through pairs of joint name and joint position. The `name` corresponds to the naming in the URDF description and the `position` is the numeric value of the joint angle (revolute joint) or the displacement (prismatic joint). In addition, `sensor_msgs/msg/JointState` contains a time stamp which is useful to synchronize nodes in particular in co-simulations. In our case a Python node publishes the joint configuration according to a simulated motion in joint space and on the ROS side RViz2 visualizes the arm configuration.

In Python you create a subscriber using `rclpy` and receive messages via a callback. A minimal publisher example for `sensor_msgs/msg/JointState` is:

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState

class JointStatePub(Node):
    def __init__(self):
        super().__init__("joint_state_pub")
        self.pub = self.create_publisher(JointState, "/joint_states", 10)

    def publish_q(self, q):
        msg = JointState()
        msg.name = [
            "joint2_to_joint1",
            "joint3_to_joint2",
            "joint4_to_joint3",
            "joint5_to_joint4",
            "joint6_to_joint5",
            "joint6output_to_joint6",
        ]
        msg.position = [float(x) for x in q]
        self.pub.publish(msg)

def main():
    rclpy.init()
    node = JointStatePub()
    # node.publish_q(q) # called inside your loop
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
  
```

Loop timing in Python can be controlled using ROS 2 timers (recommended) or a simple rate loop in the notebook when running co-simulation code.

Proceed with tasks 22 to 27 in the template.

Competences

After successfully completing the Forward Kinematics lab, students will have acquired the following competences:

- They understand the fundamental principles and mathematical foundations of forward kinematics and can derive and apply forward kinematic equations to compute the pose of a robot's end-effector from given joint variables.
- They are able to map joint space to task space and interpret this relationship both mathematically and geometrically, including how robot structure and joint parameters influence the workspace and motion capabilities.
- They can apply the Denavit-Hartenberg (DH) convention to parameterize robotic links and joints, compute DH parameters, and construct transformation matrices for serial robotic manipulators.
- They are capable of building and simulating kinematic models of multi-DOF robotic manipulators, such as the myCobot 280, using Python tooling in Jupyter notebooks (e.g., SpatialMath and Robotics Toolbox for Python).
- They can compute frame-to-frame transformations, visualize robot configurations, and implement forward kinematic models in ROS 2 environments, including URDF-based robot descriptions.
- They have practical skills in Jupyter–ROS 2 co-simulation, including publishing and subscribing to joint state information, synchronizing transformations, and visualizing robot motion in real time.
- They can integrate forward kinematics into a broader robotics software stack, linking robot description (URDF), simulation (Gazebo), and control interfaces.
- They are proficient in using ROS 2 communication mechanisms (topics, publishers, subscribers, messages) to exchange data between Jupyter/Python and ROS 2 nodes in a co-simulation context.
- They are able to critically analyze and interpret kinematic data and apply their knowledge to support advanced robotics tasks such as inverse kinematics, trajectory generation, and motion planning.

References